EXPERIMENT 14: TIC-TAC-TOE USING MINIMAX ALGORITHM

AIM

To develop a Python program for the Tic-Tac-Toe game using the Minimax Algorithm to enable the computer to select the optimal move against a human player.

OBJECTIVE

- To understand game playing in Artificial Intelligence.
- To implement the Minimax algorithm for decision-making.
- To create a two-player Tic-Tac-Toe game.
- To make the computer choose the best possible move.
- To determine the winner or draw condition.

ALGORITHM

Step 1: Start.

Step 2: Create a 3 × 3 Tic-Tac-Toe board.

Step 3: Assign X to the human player and O to the computer.

Step 4: Check whether the game has a winner or the board is full.

Step 5: Generate all possible empty positions.

Step 6: For each possible computer move, apply the Minimax algorithm.

Step 7:

- MAX player (Computer) chooses the highest score.
- MIN player (Human) chooses the lowest score.

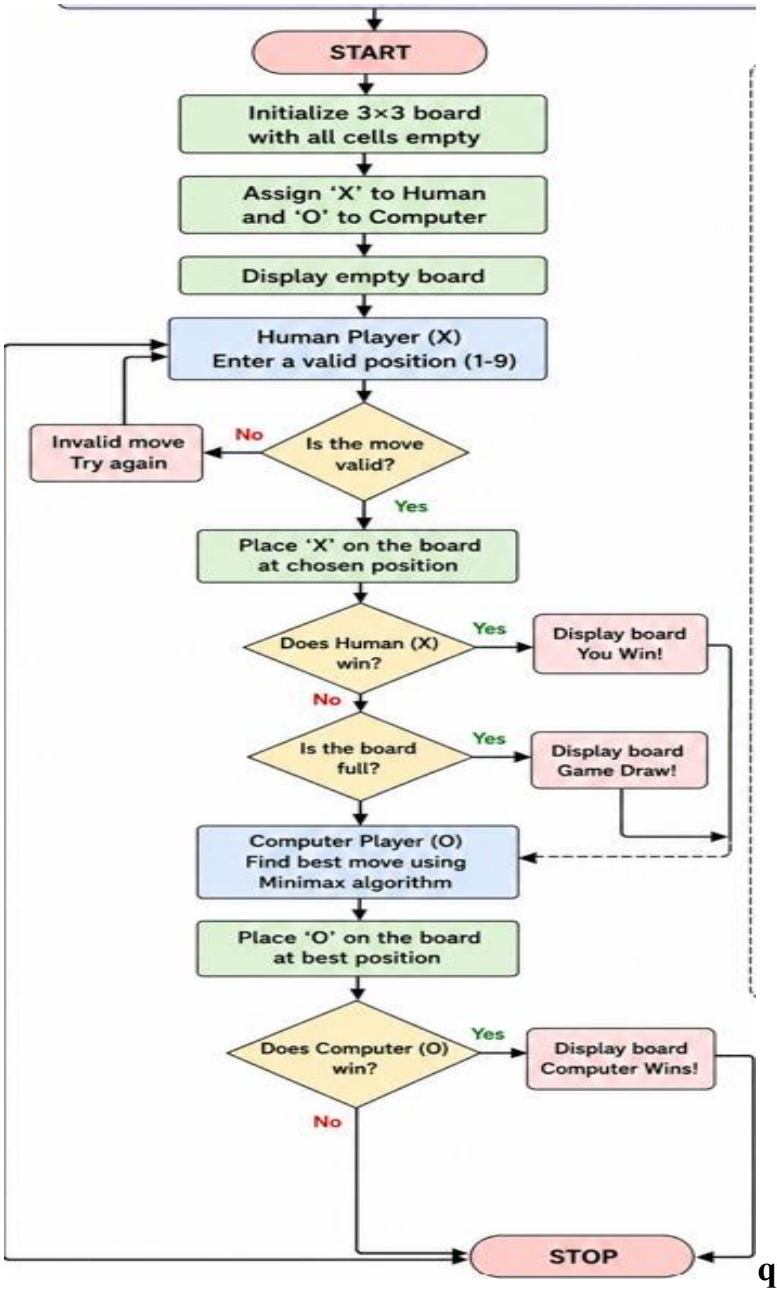
Step 8: Select the move with the best Minimax score.

Step 9: Continue until a player wins or the game ends in a draw.

Step 10: Display the final result.

Step 11: Stop.

FLOW CHART



qwj q

SOURCE CODE

```
# Tic-Tac-Toe using Minimax Algorithm


board = [" " for _ in range(9)]


def print_board():
```

```
    print()
    print(board[0], "|", board[1], "|", board[2])
    print("--+---+--")
    print(board[3], "|", board[4], "|", board[5])
    print("--+---+--")
    print(board[6], "|", board[7], "|", board[8])
    print()

def check_winner(player):
    winning_positions = [
        (0, 1, 2),
        (3, 4, 5),
        (6, 7, 8),
        (0, 3, 6),
        (1, 4, 7),
        (2, 5, 8),
        (0, 4, 8),
        (2, 4, 6)
    ]

    for a, b, c in winning_positions:
        if board[a] == board[b] == board[c] == player:
            return True

    return False

def is_draw():
```

```
    return " " not in board

def minimax(is_maximizing):
    if check_winner("O"):
        return 1

    if check_winner("X"):
        return -1

    if is_draw():
        return 0

    if is_maximizing:
        best_score = -100

        for i in range(9):
            if board[i] == " ":
                board[i] = "O"
                score = minimax(False)
                board[i] = " "
                best_score = max(best_score, score)

        return best_score

    else:
        best_score = 100
```

```
        for i in range(9):
            if board[i] == " ":
                board[i] = "X"
                score = minimax(True)
                board[i] = " "
                best_score = min(best_score, score)

        return best_score

def computer_move():
    best_score = -100
    best_move = None

    for i in range(9):
        if board[i] == " ":
            board[i] = "O"
            score = minimax(False)
            board[i] = " "

            if score > best_score:
                best_score = score
                best_move = i

    board[best_move] = "O"

print("TIC-TAC-TOE")
print("You = X, Computer = O")
```

```
while True:
    print_board()

    # Human move
    move = int(input("Enter position (1-9): ")) - 1

    if move < 0 or move > 8 or board[move] != " ":
        print("Invalid move. Try again.")
        continue

    board[move] = "X"

    if check_winner("X"):
        print_board()
        print("You Win!")
        break

    if is_draw():
        print_board()
        print("Game Draw!")
        break

    # Computer move
    computer_move()

    if check_winner("O"):
```

```
        print_board()
        print("Computer Wins!")
        break

    if is_draw():
        print_board()
        print("Game Draw!")
        break
```

OUTPUT

```
TIC-TAC-TOE
You = X, Computer = O

  |   |
--+---+--
  |   |
--+---+--
  |   |

Enter position (1-9):
```

RESULT

The Python program successfully implements the Tic-Tac-Toe game using the Minimax algorithm. The computer evaluates the possible moves and selects the optimal move, allowing it to play intelligently against the human player.